Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Proyecto Final

Importador de listas de precios de proveedores “Organizalo”

Grupo N° 174

Integrante: Franco Gabriel Vazquez

Tutor: Oscar Londero

Año 2026

Repositorio: https://github.com/fraanv1999/TPIFinalDesiderioVazquez

Versión entregada: tag v1.0-entrega-final

Sistema desplegado:

- **Backend** https://tpifinaldesideriovazquez-production.up.railway.app/api
- **Frontend**: https://wonderful-vibrancy-production-e02e7fc2.up.railway.app
- **Video final:** enlace público al video

Índice

Índice 3
Nota sobre el equipo 5
1. Problema y contexto 5
Situación que dio origen al proyecto 5
Usuarios y actores 5
Evidencia utilizada para validar el problema 5
Necesidad concreta 6
2. Objetivos y alcance 6
Objetivo general 6
Alcance implementado 6
Fuera del alcance y motivos 6
3. Análisis y modelo 7
Entidades principales 7
Reglas de negocio relevantes 9
Decisiones de modelado 10
4. Arquitectura y tecnologías 10
Integraciones 11
Seguridad, autorización y concurrencia 11
5. Implementación 11
Módulos desarrollados 11
Caso de uso principal: importar una lista 11
Dificultades técnicas y cómo se resolvieron 13
6. Pruebas y validación 14
Qué se probó 14
Casos de prueba relevantes con el caso real 14
Cómo se comprueban las reglas críticas 14
Resultados 15
Validación con usuarios 17
7. Resultados y conclusiones 17
Qué resolvió el proyecto 17
Qué cambió respecto de la situación inicial 17
Implementado y probado frente a trabajo futuro 17
Limitaciones conocidas 18
Aprendizajes 18
8. Enlaces 18

Nota sobre el equipo

El proyecto se inició con dos integrantes, Franco Vazquez y Lucas Desiderio, y así figura en las primeras entregas y en el nombre del repositorio. Lucas Desiderio dejó el trabajo después de la segunda entrega. Desde entonces el desarrollo, este informe y el video los realizó Franco Vazquez.

1. Problema y contexto

Situación que dio origen al proyecto

El negocio es una ferretería “todo en uno”: además de ferretería vende artículos de plomería, gas y pinturas, y trabaja como mayorista con unos diez proveedores. Cada proveedor envía su lista de precios en Excel y, entre todos, llegan dos o tres listas nuevas por semana. Cada proveedor usa su propio formato: hojas distintas, encabezados en filas distintas, columnas con otros nombres, títulos de categoría mezclados con los productos, celdas combinadas y fórmulas propias.

El sistema de gestión del negocio solo acepta un archivo con columnas fijas. Hoy cada lista se pasa a mano a ese formato y se aplican los cálculos de precio (ingresos brutos, ganancia, IVA). Eso lleva entre 2 y 3 horas por semana y genera errores en precios y códigos.

Usuarios y actores

Actor	Rol
Operador del negocio	Recibe las listas, las importa, revisa la vista previa, confirma y descarga el archivo para el sistema de gestión.
Administrador	Además administra los proveedores.
Proveedores	Envían las listas en Excel. No usan el sistema.
Sistema de gestión del negocio (destino)	Sistema externo que recibe el archivo final mediante su importador de productos. Organizalo no lo modifica.

Evidencia utilizada para validar el problema

- **Listas reales de proveedores** (carpeta dataset-example/dataset-providers): Ferretería (hoja LSTPRE), Paint Roller y el libro LISTAS CUENTA 2, que trae siete hojas con estructuras distintas. Las versiones que el negocio recibió de Ferretería y Paint Roller entre julio y noviembre están en dataset-example/dataset-versiones, una carpeta por fecha de recepción.
- **Planillas que se completaban a mano** para la plataforma anterior (dataset-example/dataset-previous-platform): Kallay, Bulones y Matienzo, con los cálculos de precio hechos en Excel.
- **Par original y corregido** de Paint Roller del 02/10: la lista tal como llegó y la corrección manual que hizo el negocio. Se usó como prueba de referencia.
- **Conversaciones con el negocio** sobre el tiempo dedicado y los errores frecuentes. El cliente explicó que cada cambio de precio de un proveedor lo obliga a tomar un Excel con un formato difícil de entender y pasarlo a mano al sistema de gestión, una tarea que se repite todas las semanas. Organizalo surge para cubrir ese paso intermedio entre la lista del proveedor y el sistema del negocio.
- **Devoluciones del tutor** (15/08, 25/08 y 29/09), que acotaron el problema y pidieron trabajar con casos reales.

Necesidad concreta

Transformar la lista de cada proveedor al formato del sistema de gestión de forma repetible, con los cálculos de cada proveedor, y saber con certeza qué filas quedaron afuera y por qué. Además, detectar cuándo un proveedor cambia el formato de su lista, en lugar de cargar datos equivocados sin aviso.

2. Objetivos y alcance

Objetivo general

Desarrollar Organizalo, un sistema web que importe listas de precios en Excel de distintos proveedores, las transforme según un perfil configurado una vez por cada hoja y genere un archivo listo para el importador de productos del sistema de gestión del negocio, con un reporte de las filas rechazadas.

Alcance implementado

- Perfiles de importación por hoja de proveedor: hoja, fila de encabezados, mapeo de columnas, transformaciones y reglas de cálculo.
- Control de estructura: compara el archivo con el perfil y rechaza la importación si cambió el formato.
- Clasificación de cada fila en válida, ignorada (vacía o título) o con error, con el motivo.
- Cálculos de precio en orden y sin redondeo hasta la exportación.
- Vista previa, confirmación o cancelación de cada importación e historial.
- Exportación del XLSX para el sistema de gestión y del reporte de errores y avisos.
- Alta, edición y baja de proveedores.
- Login con usuarios y roles (ADMIN y OPERADOR).
- Frontend en React conectado a la API.
- Despliegue de la API, la base de datos y el frontend en Railway.

Fuera del alcance y motivos

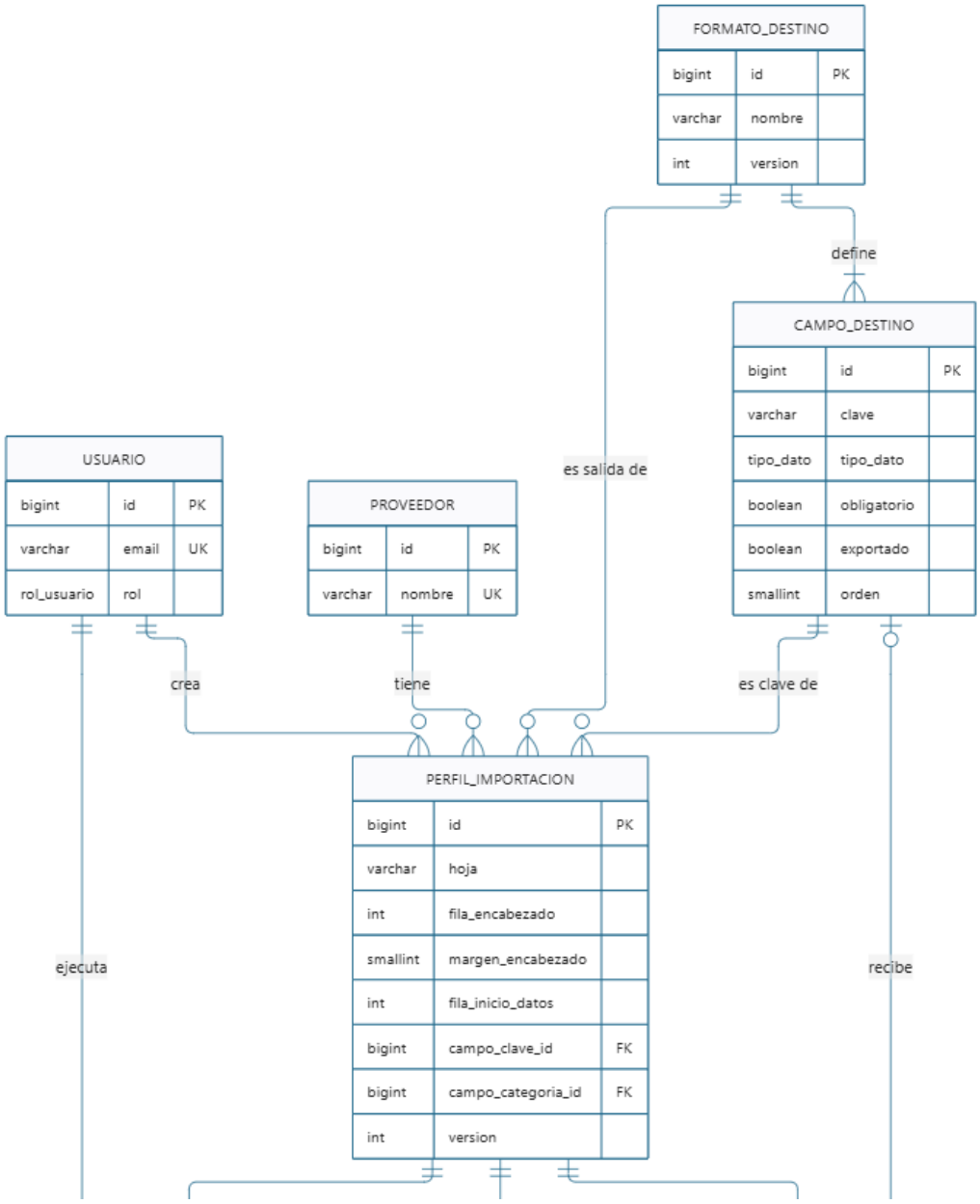
Funcionalidad	Motivo del recorte
Stock, ventas y multitenancy	Por indicación del tutor (15/08): el objetivo es resolver la importación, no construir otro sistema de gestión.
Conversión de monedas y unidades	Las listas en dólares se tratan como fuente no soportada. Un cambio a dólares se rechaza por encabezado modificado.
Matching de productos entre proveedores	Requiere reglas de negocio que no estaban definidas. Queda como evolución futura.
Edición de perfiles desde la interfaz	Los perfiles se cargan en la base (seed.sql) y la interfaz los muestra. Se priorizó el motor y las pruebas.
Importar archivos que ya vienen en el formato destino	Módulo opcional de la 2.ª entrega que no se implementó.
Purga automática de importaciones viejas	Se guarda la fecha expira_en, pero no hay una tarea que borre.

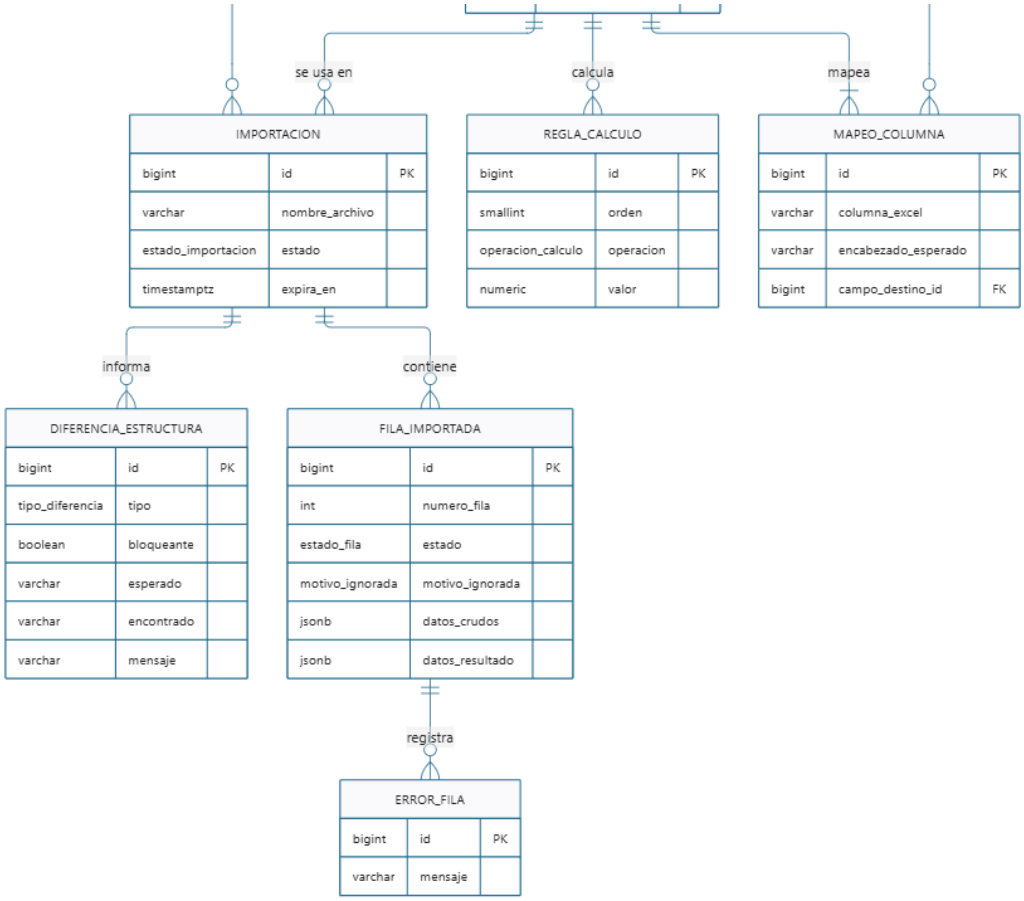
3. Análisis y modelo

Entidades principales

Entidad	Qué representa
Usuario	Persona que usa el sistema, con rol ADMIN u OPERADOR.
Proveedor	Quien envía las listas. Solo nombre y estado.
Formato destino y campos	Columnas que acepta el sistema de gestión del negocio y su tipo.
Perfil de importación	Contrato de una hoja de un proveedor: hoja, fila de encabezados, campo clave, campo de categoría y versión.
Mapeo de columna	Columna del Excel → campo destino, con el encabezado esperado y sus transformaciones.
Regla de cálculo	Operación ordenada sobre los precios (sumar porcentaje, multiplicar, etc.).
Importación	Un archivo procesado: estado, usuario, fecha y resumen.
Diferencia de estructura	Algo del archivo que no coincide con el perfil.
Fila importada y error de fila	Cada fila leída con sus datos originales y transformados, su estado y el motivo de rechazo.

Diagrama entidad-relación





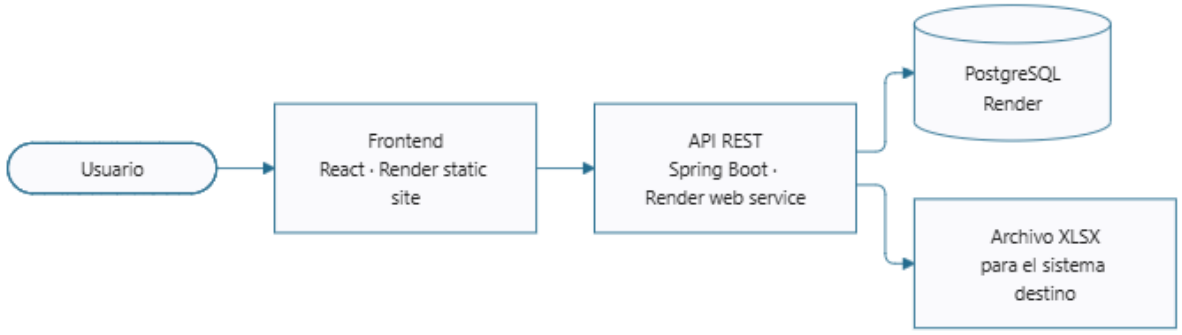
Reglas de negocio relevantes

- **Control de estructura:** los encabezados se comparan sin distinguir mayúsculas, tildes ni espacios. Una hoja inexistente, un encabezado modificado, una columna faltante o una columna en otra posición bloquean la importación. Un encabezado desplazado unas filas, dentro del margen del perfil, es solo un aviso.
- **Clasificación de filas:** si no hay valores, la fila es VACÍA. Si no tiene código ni números, es un TÍTULO y su texto pasa a ser la categoría de los productos siguientes. Si falta un dato obligatorio o un valor no se puede convertir, es un ERROR. El resto es VÁLIDA.
- **Un 0 cuenta como vacío** en campos numéricos, porque algunas listas (Fenix) arrastran fórmulas que dan 0 en filas sin producto.
- **Porcentajes como fracción** en todas las capas: 0.04 = 4 %.
- **Redondeo solo al exportar:** el motor conserva la precisión completa y el archivo final sale con 2 decimales.
- **Estados de la importación:** PREVIEW → CONFIRMADA o CANCELADA, o RECHAZADA si falla la estructura. Solo se exporta una importación confirmada.
- **Una fila con error no aporta datos parciales:** se informa en el reporte y queda afuera del archivo final.

Decisiones de modelado

- **Un perfil por hoja**, no por proveedor: un mismo libro puede traer varias listas con estructuras distintas.
- **PostgreSQL con una columna JSONB** para la fila original, en lugar de MongoDB. Los datos tienen estructura fija y relaciones; lo único variable es la fila del Excel. Lo cuestionó el tutor en la 1.ª devolución.
- **Operaciones de cálculo cerradas** (un tipo enumerado), sin fórmulas libres, para que sean verificables.
- **Versiones de perfil:** si el proveedor cambia la lista se crea una versión nueva y el historial no se toca.

4. Arquitectura y tecnologías



Componente	Responsabilidad	Por qué se eligió
Frontend: React + Tailwind (Vite)	Login, proveedores, perfiles, asistente de importación, historial y descargas.	Experiencia previa del equipo y desarrollo rápido de pantallas.
Backend: Java 17 + Spring Boot	API REST, motor de transformación, seguridad y generación de archivos.	Ecosistema maduro para APIs, seguridad y pruebas. Además, el lenguaje de la carrera.
Apache POI	Leer los Excel de los proveedores y escribir los XLSX de salida.	Es la librería estándar en Java para XLSX y soporta celdas combinadas y fórmulas.
PostgreSQL	Usuarios, proveedores, perfiles, importaciones, filas y errores.	Datos relacionales con integridad. JSONB para la fila original.
JDBC (JdbcClient), sin JPA	Acceso a datos.	El esquema usa ENUM, arrays y JSONB, así schema.sql es la única definición de las tablas.
Docker + Railway	Despliegue de la API, la base de datos y el frontend.	Railway construye la imagen de la API desde el Dockerfile, provee PostgreSQL administrado y publica el frontend como sitio estático en otro servicio del mismo proyecto.

Integraciones

La integración con el sistema de gestión del negocio es por archivo: Organizalo genera un XLSX con las columnas de su importador de productos (Código, Nombre, Descripción, Categoría, Stock, Precio, Ingresos brutos, Ganancia, IVA y Precio + IVA). No hay integración por API.

Seguridad, autorización y concurrencia

- **Autenticación:** HTTP Basic con contraseñas guardadas con BCrypt. No hay sesión en el servidor. En producción viaja sobre HTTPS.
- **Autorización:** cualquier usuario activo importa y exporta; solo ADMIN administra proveedores.
- **CORS** limitado a los orígenes configurados en CORS_ORIGENES.
- **Secretos:** la conexión a la base y los orígenes se configuran con variables de entorno. No hay credenciales en el repositorio.
- **Estados:** confirmar una importación que no está en PREVIEW, o que no tiene filas válidas, devuelve 409 Conflict.

5. Implementación

Módulos desarrollados

Módulo	Qué hace
Usuarios y seguridad	Login, roles e identificación del usuario en cada importación.
Proveedores	Alta, edición y baja. Si tiene perfiles, se desactiva en lugar de borrarse.
Perfiles de importación	Lectura desde la base de cada perfil con sus columnas, transformaciones y reglas.
Lectura de Excel	Lee la hoja, las filas y los valores de las celdas.
Motor de transformación	Control de estructura, clasificación, conversión, transformaciones y cálculos.
Importaciones	Vista previa persistida, confirmación, cancelación e historial.
Exportación	XLSX para el sistema de gestión y XLSX de errores y avisos.
Frontend	Pantallas para operar todo lo anterior.

Caso de uso principal: importar una lista

1. El operador inicia sesión.
2. Elige el perfil (por ejemplo, Ferretería · LSTPRE) y sube el Excel.
3. El sistema controla la estructura. Si no coincide, la importación queda RECHAZADA y muestra cada diferencia.
4. Si coincide, clasifica y transforma cada fila, y muestra la vista previa con válidas, ignoradas y errores.
5. El operador confirma (o descarta) la importación.

6. Descarga el XLSX para el sistema de gestión y, si hace falta, el reporte de errores.

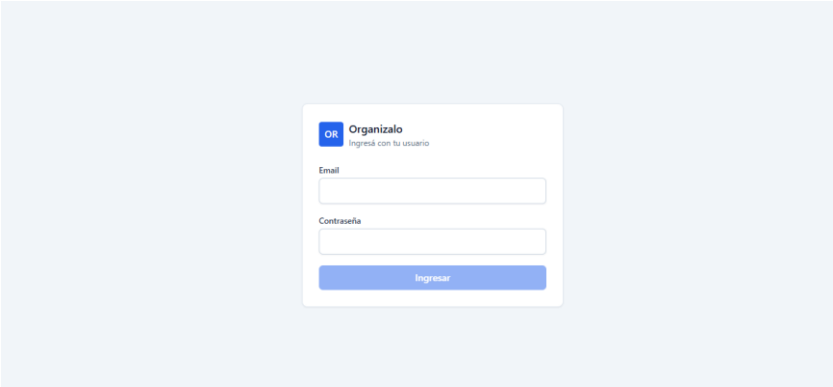


Figura 1. Inicio de sesión.

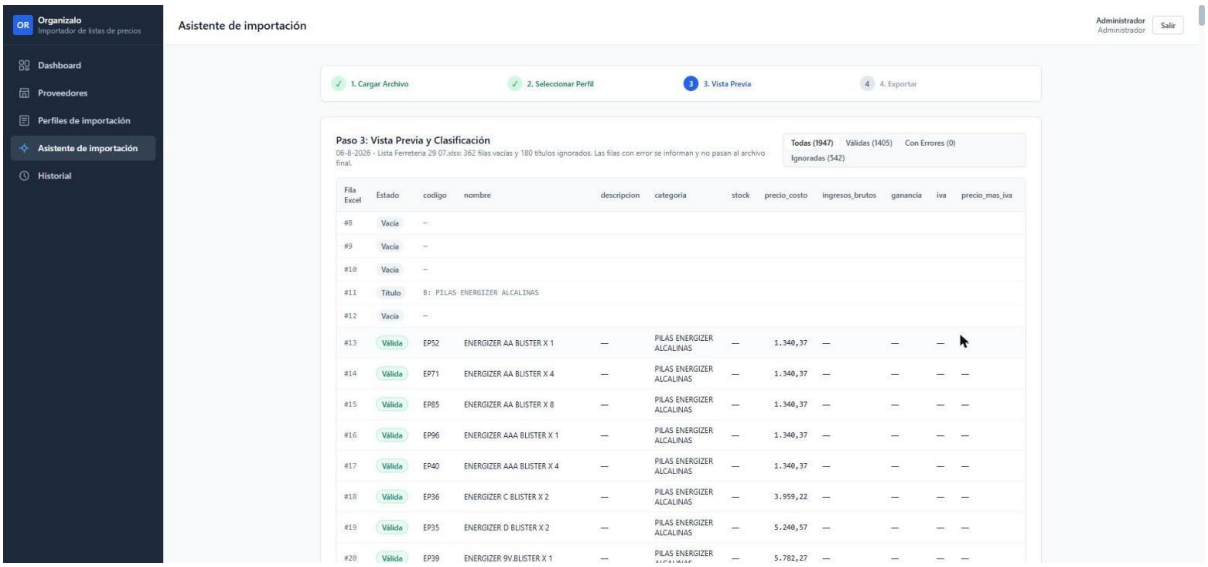


Figura 2. Vista previa de Lista Ferreteria 29 07.xlsx en el sistema desplegado: estado de cada fila, categoría tomada del título y precio de costo.

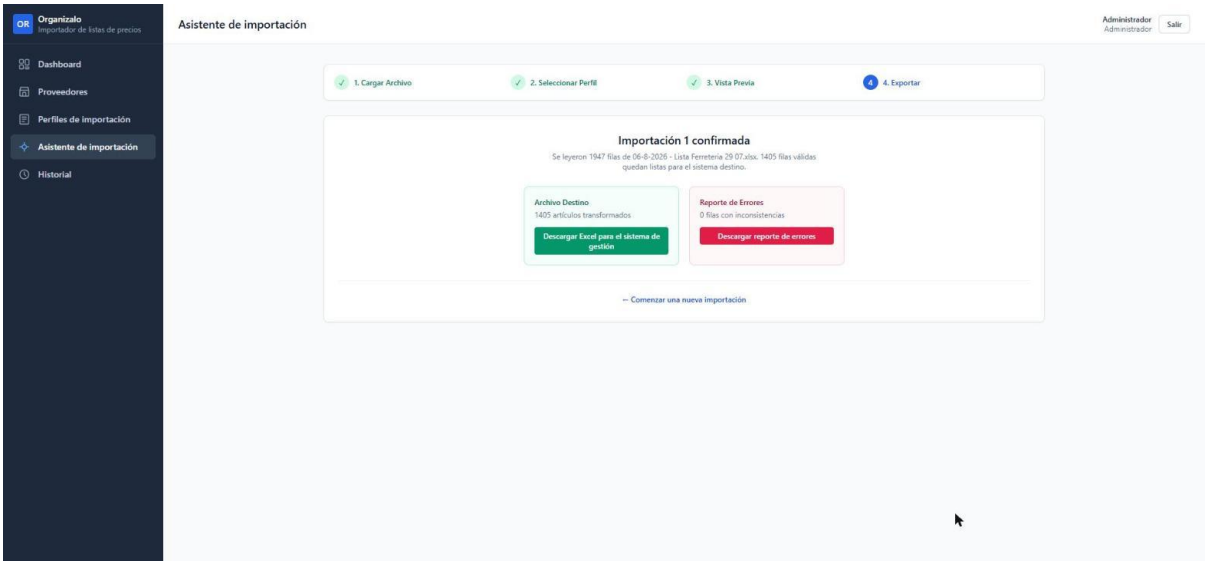


Figura 3. Importación confirmada en el sistema desplegado: 1405 artículos para el sistema de gestión y el reporte de errores.

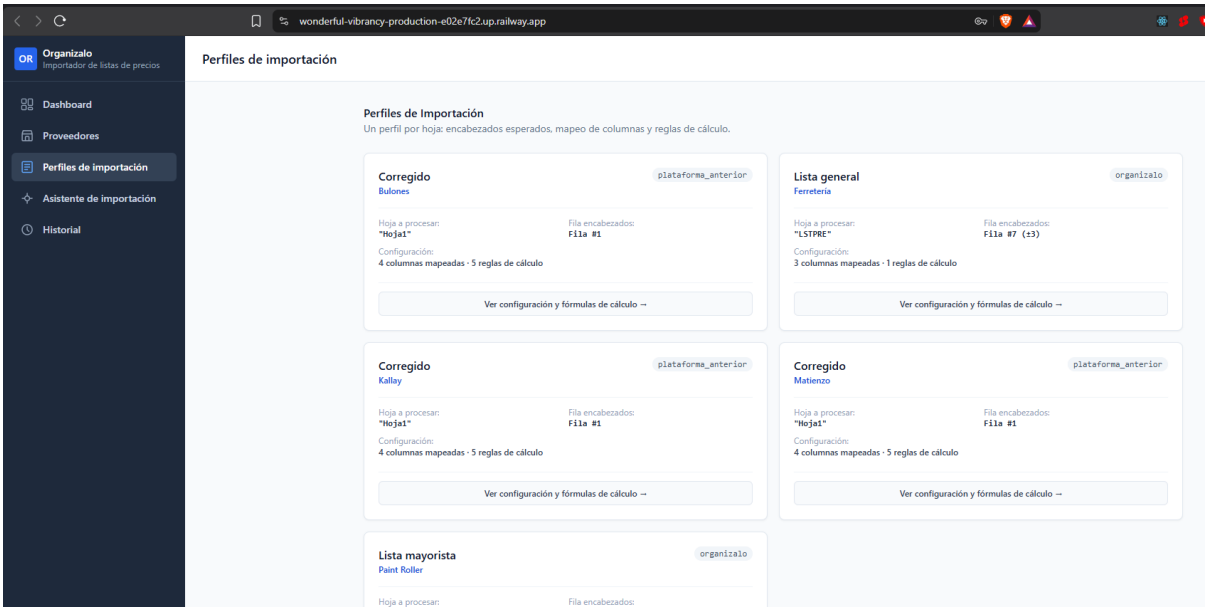


Figura 4. Perfiles de importación en el sistema desplegado en Railway.

Dificultades técnicas y cómo se resolvieron

Dificultad	Solución
LSTPRE trae los encabezados en la fila 7 o en la 8 según la versión.	Campo margen_encabezado: se aceptan los mismos encabezados unas filas más arriba o más abajo, con un aviso.
Paint Roller pone el nombre del producto una vez por grupo, en celdas combinadas.	Transformación RELLENAR_ABAJO, que se reinicia en cada fila título.
Fenix arrastra fórmulas que dan 0 en filas sin producto.	Un 0 en un campo numérico cuenta como vacío al clasificar.
Hojas en dólares mezcladas con listas en pesos.	Se tratan como fuente no soportada y un cambio de moneda se rechaza por encabezado modificado.
Códigos repetidos dentro de una lista (RAPIFIX aparece 49 veces).	Se informan como aviso en el reporte.
JSONB no conserva el orden de las claves.	Al leer una importación, los campos se ordenan según el formato destino.
Cambio de MongoDB a PostgreSQL después de la 1.ª devolución.	Modelo relacional con una columna JSONB para la fila original.
Despliegue en Railway: la API y el frontend están en el mismo repositorio, y el servicio del frontend tomaba el railway.json de la API. El primer build del frontend falló. En total llevó unos dos días de configuración.	Cada servicio con su configuración: el frontend con Root Directory frontend y su propio /frontend/railway.json, que deja el build a Railpack. Variables separadas por servicio: DB_* referenciadas a PostgreSQL y CORS_ORIGENES en la API; VITE_API_URL en el frontend, que se aplica al construir, así que cambiarla exige redesplegar.

6. Pruebas y validación

Qué se probó

Tipo	Clases	Qué comprueban
Unitarias del motor	ConversorTest, CalculadoraTest, ControlEstructuraTest, MotorTransformacionTest	Conversión de valores, operaciones de cálculo, diferencias de estructura y clasificación de filas, con datos armados en memoria.
De referencia con Excel reales	PreviewLstpreTest, PlataformaAnteriorTest, ParPaintRollerTest, VersionesListasTest	El motor contra las listas reales y contra los cálculos hechos a mano.
De API	ImportacionesApiTest, ExportacionApiTest, ProveedoresApiTest, SeguridadApiTest	Endpoints, estados de la importación, archivos exportados, roles y login.
De base de datos	ScriptsSqlTest, InicializadorBaseTest, RepositorioPerfilesTest	Que los scripts SQL corran, la creación automática del esquema y la lectura de perfiles.

Las pruebas usan JUnit 5, AssertJ y MockMvc, con un PostgreSQL embebido, así que no hace falta tener una base instalada.

Casos de prueba relevantes con el caso real

Prueba	Resultado esperado y obtenido
Clasificación de Lista Ferreteria 19 08.xlsx (LSTPRE)	1412 filas válidas, 180 títulos, 361 vacías y 0 errores.
Par original y corregido de Paint Roller (02/10)	El resultado del motor coincide con la corrección manual del negocio.
Cálculos de Kallay, Bulones y Matienzo	Las reglas de cada perfil dan los mismos valores que calculó Excel. En Bulones, las filas 216 a 218 tienen el valor escrito a mano sobre la fórmula, y el motor las detecta como distintas.
Versiones reales de Ferretería y Paint Roller (julio a noviembre)	El mismo perfil procesa las 7 versiones de Ferretería y las 4 de Paint Roller. Una corrección manual con otros encabezados se rechaza con 6 diferencias. Detalle en la sección siguiente.

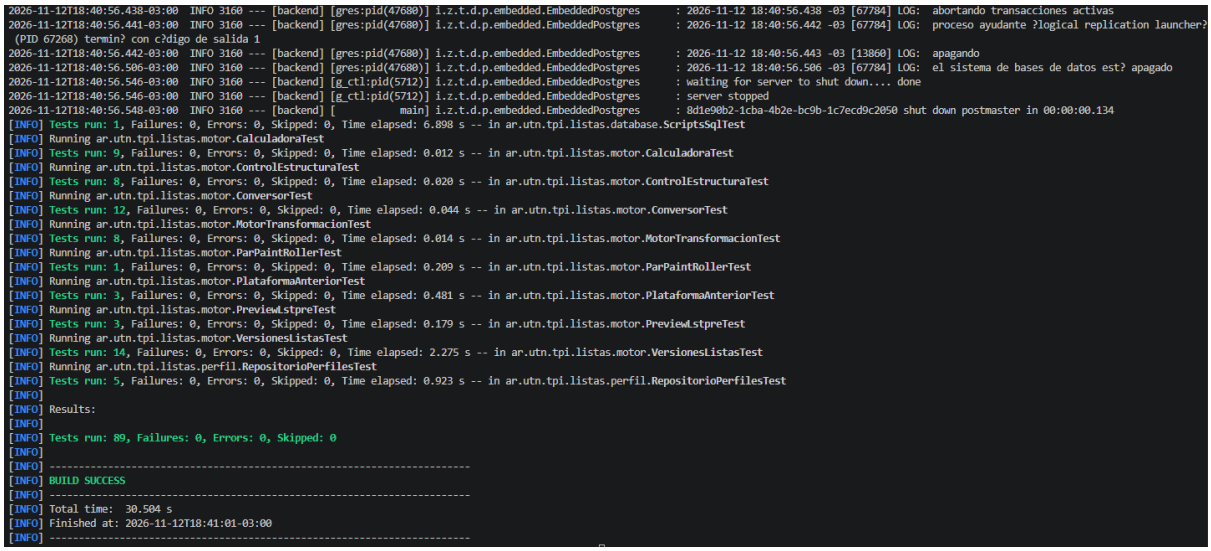
Cómo se comprueban las reglas críticas

- **Cambio de formato del proveedor:** ControlEstructuraTest cubre cada tipo de diferencia y verifica que las bloqueantes rechacen la importación sin procesar filas.
- **Cálculos de precio:** CalculadoraTest prueba cada operación y PlataformaAnteriorTest compara contra planillas reales.

- **Estados:** ImportacionesApiTest verifica que solo se confirme una vista previa con filas válidas (409 en los demás casos) y que no se exporte una importación sin confirmar.
- **Seguridad:** SeguridadApiTest verifica que sin credenciales se responda 401 y que un OPERADOR no pueda administrar proveedores.

Resultados

Ejecución de ./mvnw test del 12/11/2026: **89 pruebas, 0 fallas, 0 errores, 0 salteadas**. Todos los Excel que usan las pruebas están en dataset-example/, incluidas las versiones de Ferretería y Paint Roller, así que cualquiera que clone el repositorio puede repetirlas sin archivos externos.



Reutilización del perfil con listas nuevas

El tutor pidió comprobar que un perfil se puede reutilizar con listas nuevas del mismo proveedor. Se corrieron los perfiles de Ferretería y Paint Roller, sin cambios, sobre todas las versiones reales que el negocio recibió entre julio y noviembre. Las tres que también se subieron al sistema desplegado dieron los mismos números. Debajo de cada lista figura su archivo dentro de dataset-example/dataset-versiones, en el repositorio. La prueba VersionesListasTest verifica esta tabla en cada ejecución. El detalle del recorrido, con el archivo subido en cada paso, está en docs/prueba-sistema-desplegado.md.

Lista	Válidas / errores	Observación
Ferretería 29/07 06-8-2026/Lista Ferreteria 29 07.xlsx	1405 / 0	Probada también en el sistema desplegado y confirmada.
Ferretería 19/08 19-8-2026/Lista Ferreteria 19 08.xlsx	1412 / 0	Versión usada para armar el perfil.
Ferretería 07/09 08-9-2026/Lista Ferreteria 07 09.xlsx	1403 / 0	Encabezados en la fila 8 en vez de la 7: aviso, dentro del margen ±3.
Ferretería 21/09 22-9-2026/Lista Ferreteria 21 09.xlsx	1396 / 0	Encabezados en la fila 8: aviso.
Ferretería 05/10 06-10-2026/ y 13-10-2026/Lista Ferreteria 05 10.xlsx	1392 / 0	

Lista	Válidas / errores	Observación
Ferretería 16/10 26-10-2026/Lista Ferreteria 16 10.xlsx	1394 / 0	
Ferretería 02/11 02-11-2026/Lista Ferreteria 02 11.xlsx	1383 / 0	
Paint Roller 05/09 05-9-2026/Lista de precios Mayorista - PAINT ROLLER.xlsx	168 / 2	Filas 66 y 81: precio sin código.
Paint Roller 02/10 02-10-2026/Lista de precios Mayorista - PAINT ROLLER.xlsx	166 / 4	Versión usada para armar el perfil y comparada con la corrección manual.
Paint Roller 29/10 29-10-2026/Lista de precios Mayorista - PAINT ROLLER.xlsx	161 / 9	Además de 66 y 81, siete productos con “SIN STOCK” en el precio. Probada también en el sistema desplegado.
Paint Roller 02/11 02-11-2026/Lista de precios Mayorista - PAINT ROLLER.xlsx	159 / 11	Nueve productos con “SIN STOCK”. Probada también en el sistema desplegado.
Paint Roller, corrección manual del 15/08 15-8-2026/Lista de precios Mayorista N°42 - PAINT ROLLERCORRECION - Con medidas.xlsx	Rechazada	Otros encabezados (NOMBRE, MEDIDAS, PRECIO) y faltan Bulto, Aclaración y Última actualización: 6 diferencias bloqueantes.

Todas las versiones de Ferretería se procesan sin errores. Los errores de Paint Roller son datos que faltan en la lista del proveedor, y cada uno se informa con su motivo. El archivo que no cumple el contrato se rechaza sin procesar filas; lo mismo pasa si se elige un perfil equivocado (figura 5).

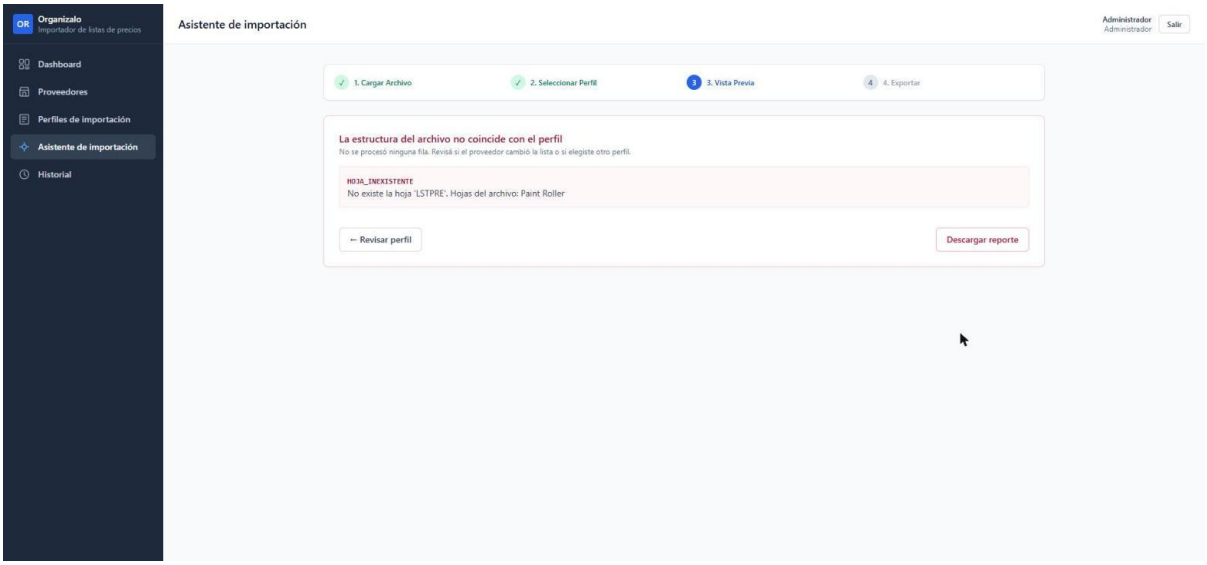


Figura 5. Lista de Paint Roller procesada con el perfil de Ferretería en el sistema desplegado: se rechaza y no se procesa ninguna fila.

Validación con usuarios

Se le mostró Organizalo al cliente con sus propias listas. Le pareció adecuado para lo que necesita: le saca de encima la carga manual de cada semana, que era el problema principal. Quiere sumarle funciones más adelante, pero esta versión (un MVP) ya resuelve ese problema.

Además, la validación técnica se hizo con los archivos reales del negocio y con las correcciones manuales que había hecho. La prueba de punta a punta se hizo en el sistema desplegado en Railway: se importó la lista de Ferretería del 29/07 con su perfil, se leyeron 1947 filas (1405 válidas, 362 vacías, 180 títulos y ningún error), se confirmó la importación y quedó disponible el Excel para el sistema de gestión (figuras 2 y 3). El recorrido completo, con capturas de cada pantalla, está en docs/prueba-sistema-desplegado.md.

7. Resultados y conclusiones

Qué resolvió el proyecto

Una lista de un proveedor configurado se transforma con Organizalo al formato del sistema de gestión en minutos y no en horas, con los mismos cálculos que se hacían a mano y con un reporte de cada fila que quedó afuera. Si el proveedor cambia el formato, el sistema lo detecta y dice qué cambió, en lugar de cargar datos equivocados.

Qué cambió respecto de la situación inicial

Antes	Ahora
Copiar columna por columna a mano.	Subir el archivo y confirmar la vista previa.
Errores de precio y código que se descubren tarde.	Cada fila con error se informa con el motivo.
Un cambio de formato pasa desapercibido.	La importación se rechaza y se lista cada diferencia.
Los cálculos dependen de la planilla de cada uno.	Reglas por proveedor, guardadas y probadas.

Implementado y probado frente a trabajo futuro

Implementado y probado	Trabajo futuro
Motor con control de estructura, clasificación y cálculos	Editor de perfiles desde la interfaz
Persistencia de importaciones e historial	Importar archivos que ya vienen en el formato destino
Exportación para el sistema de gestión y reporte de errores	Tarea de purga de importaciones vencidas
Proveedores, login y roles	Conversión de monedas y matching de productos entre proveedores
API, base de datos y frontend desplegados en Railway	Rediseño del asistente paso a paso para que sea más intuitivo para el usuario final

Limitaciones conocidas

- El negocio confirmó los porcentajes de Ferretería y Paint Roller (ingresos brutos 4 %, ganancia 45 % e IVA 21 %), pero en la versión entregada esos dos perfiles todavía usan la regla provisoria precio = costo. Cargar esas reglas es el próximo paso.
- En la lista LSTPRE, TIP_LST indica el tipo de precio (precio de lista) y CON_IVA el precio con IVA. El sistema lee esas columnas, pero no las usa para calcular. Falta confirmar si el IVA de Bulones sobre el costo es intencional.
- Solo se aceptan listas en pesos.
- En la sección ACCESORIOS de Paint Roller, la columna Codigo trae la marca (RAPIFIX, FIJAPEL) y no un código de producto: hasta 49 filas con el mismo código. El sistema lo informa como aviso, pero esas filas pasan al archivo final. Falta definir con el negocio qué código usar.
- Los productos con “SIN STOCK” en el precio se informan como error. El negocio podría preferir marcarlos como sin stock.
- Los perfiles nuevos se cargan por SQL, no desde la interfaz.
- Los servicios de Railway tienen activado el modo Serverless: después de unos minutos sin pedidos se suspenden, y el primer pedido siguiente tarda varios segundos mientras la API vuelve a arrancar.

Aprendizajes

Las devoluciones del tutor sirvieron para saber dónde enfocar el proyecto y recortar el alcance a algo que se pudiera terminar y defender. Después de la salida de Lucas Desiderio, la prioridad fue simplificar: quedarse con lo que resuelve el problema del cliente (el motor de transformación, el control de estructura y la exportación) e identificar bien las fortalezas de la herramienta, para que también pueda servir a otros comercios con el mismo problema.

Trabajar desde temprano con los archivos reales del negocio cambió el modelo (margen de encabezado, rellenar hacia abajo, ceros como vacíos), y las pruebas contra las correcciones manuales dieron confianza para cambiar el código sin romper los cálculos.

Si empezara de nuevo, pondría antes el foco en la experiencia de uso: rediseñaría el asistente paso a paso para que el usuario final lo entienda desde el primer uso, sin explicación previa.

8. Enlaces

Recurso	Enlace
Repositorio	https://github.com/fraanv1999/TPIFinalDesiderioVazquez
Versión entregada	tag v1.0-entrega-final
Sistema desplegado (API)	https://tpifinaldesideriovazquez-production.up.railway.app/api/salud
Sistema desplegado (frontend)	https://wonderful-vibrancy-production-e02e7fc2.up.railway.app
Video final	enlace público
Cuenta de demostración	operador@tpi.local / operador123 (usuario de prueba de seed.sql)